

Fundamentos teóricos

Caminos de costo mínimo en grafos y el algoritmo de Dijkstra

Resumen

Este documento presenta de forma rigurosa los fundamentos teóricos sobre los que se construye el simulador de grafos: la teoría de grafos, el problema del camino más corto, el algoritmo de Dijkstra (con su pseudocódigo, demostración de correctitud y análisis de complejidad), su variante hacia un destino, y la comparación con los algoritmos de Bellman-Ford y Floyd-Warshall.

Contenido

1. Introducción y motivación
2. Fundamentos de teoría de grafos
3. Representación: matriz de adyacencia y listas
4. El problema del camino más corto
5. Subestructura óptima y relajación de aristas
6. Algoritmo de Dijkstra
7. Demostración de correctitud
8. Análisis de complejidad
9. Caminos hacia un destino (grafo inverso)
10. Comparación: Bellman-Ford y Floyd-Warshall
11. Aplicaciones y relación con la implementación
12. Glosario y referencias

1. Introducción y motivación

Numerosos problemas de la ingeniería y de la vida cotidiana pueden modelarse mediante **grafos**: redes de transporte, redes de computadoras, planificación de tareas, circuitos, redes sociales o sistemas de navegación. Uno de los problemas más estudiados es el del **camino de costo mínimo**: dado un punto de partida, hallar la ruta más “barata” (en distancia, tiempo o costo) hacia uno o todos los demás puntos de la red.

El algoritmo de **Dijkstra**, propuesto por el científico neerlandés Edsger W. Dijkstra en 1959, resuelve eficientemente este problema cuando los costos de las conexiones no son negativos. Constituye un ejemplo paradigmático de la técnica de diseño **voraz** (*greedy*) y es la base teórica del simulador desarrollado en este proyecto.

2. Fundamentos de teoría de grafos

Definición 2.1 — Grafo.

Un grafo es un par $G = (V, E)$, donde V es un conjunto finito no vacío de **vértices** (o nodos) y E es un conjunto de **aristas** (o arcos) que relacionan pares de vértices. Si las aristas son pares ordenados (u, v) el grafo es **dirigido** (dígrafo); si son pares no ordenados $\{u, v\}$, es **no dirigido**.

Definición 2.2 — Grafo ponderado.

Un grafo ponderado (o etiquetado) añade una función de peso $w : E \rightarrow \mathbb{R}$ que asigna a cada arista un número real, su **costo**. En este trabajo se exige $w(u, v) \geq 0$ (pesos no negativos), condición necesaria para Dijkstra.

Conceptos asociados:

- **Adyacencia**: dos vértices son adyacentes si existe una arista que los une.
- **Grado**: número de aristas incidentes en un vértice. En dígrafos se distingue grado de entrada y de salida.
- **Camino (o ruta)**: secuencia de vértices $v_0, v_1, \dots, v_k$ tal que cada par consecutivo está unido por una arista. Su **longitud (costo)** es la suma de los pesos de sus aristas.
- **Ciclo**: camino que empieza y termina en el mismo vértice.
- **Conexidad**: un grafo es conexo si existe un camino entre cualquier par de vértices.

3. Representación: matriz de adyacencia y listas

Para procesar un grafo en una computadora se necesita representarlo. Las dos formas clásicas son:

3.1 Matriz de adyacencia

Es una matriz cuadrada M de tamaño $n \times n$ (con $n = |V|$). Para un grafo ponderado se define como **matriz de costos**:

$M[i][j] = 0$	si $i = j$
$M[i][j] = w(i, j)$	si existe la arista $i \rightarrow j$
$M[i][j] = \text{infinito}$	si no existe arista (en el código se usa -1)

Ocupa un espacio de $O(n^2)$. Es ideal para grafos **densos** y permite consultar la existencia y el peso de una arista en tiempo constante $O(1)$. El simulador usa esta representación al importar/exportar la matriz, siguiendo la convención de la guía de utilizar **-1** para indicar la ausencia de arista.

3.2 Lista de adyacencia

Cada vértice guarda una lista de sus vecinos y los pesos correspondientes. Ocupa $O(V + E)$ y es preferible para grafos **dispersos** (con pocas aristas). Internamente, el simulador mantiene una lista de aristas {origen, destino, peso}, equivalente a esta representación.

4. El problema del camino más corto

Dado un grafo ponderado, el problema admite varias formulaciones:

- **Fuente única (SSSP):** hallar el camino mínimo desde un vértice origen s hacia todos los demás. Es lo que resuelve Dijkstra.
- **Par único:** el camino mínimo entre dos vértices concretos s y t (caso particular del anterior).
- **Todos los pares (APSP):** el camino mínimo entre cada par de vértices (lo resuelve Floyd-Warshall).

Definición 4.1 — Distancia mínima.

Se denota $\text{delta}(s, v)$ al costo del camino de menor peso entre s y v . Si no existe ningún camino de s a v , se define $\text{delta}(s, v) = \text{infinito}$. El objetivo del SSSP es calcular $\text{delta}(s, v)$ para todo v en V , junto con un camino que lo realice.

5. Subestructura óptima y relajación de aristas

Principio de optimalidad (subestructura óptima)

Todo subcamino de un camino mínimo es, a su vez, un camino mínimo. Es decir, si el camino más corto de s a v pasa por un vértice w , entonces la porción de s a w y la de w a v son también caminos mínimos entre esos extremos.

Esta propiedad es la que habilita el enfoque incremental. La operación elemental que la explota se llama **relajación** de una arista (u, v) :

```
relajar(u, v):  
    si dist[u] + w(u, v) < dist[v]:  
        dist[v] = dist[u] + w(u, v)  
        prev[v] = u           # u es el predecesor de v
```

Relajar comprueba si llegar a v pasando por u es más barato que la mejor estimación conocida; si lo es, se actualiza la distancia y se recuerda el predecesor (lo que permite reconstruir la ruta al final).

6. Algoritmo de Dijkstra

Dijkstra construye las distancias mínimas de forma **voraz**: en cada paso fija definitivamente la distancia del vértice no visitado más cercano al origen, y relaja sus aristas salientes. La intuición es la de una “mancha” que se expande siempre por el borde de menor costo.

6.1 Pseudocódigo

```
Dijkstra(G, w, s):
  para cada v en V:
    dist[v] = infinito; prev[v] = NULO
  dist[s] = 0
  Q = todos los vertices de V          # cola de prioridad por dist
  mientras Q no este vacia:
    u = extraer_minimo(Q)              # vertice con menor dist
    para cada arista (u, v) saliente de u:
      relajar(u, v)                    # actualiza dist[v] y prev[v]
  devolver dist, prev
```

6.2 Ejecución sobre el grafo de la guía

Se aplica el algoritmo al dígrafo ponderado de la Figura 1, tomando como origen el vértice $s = 1$.

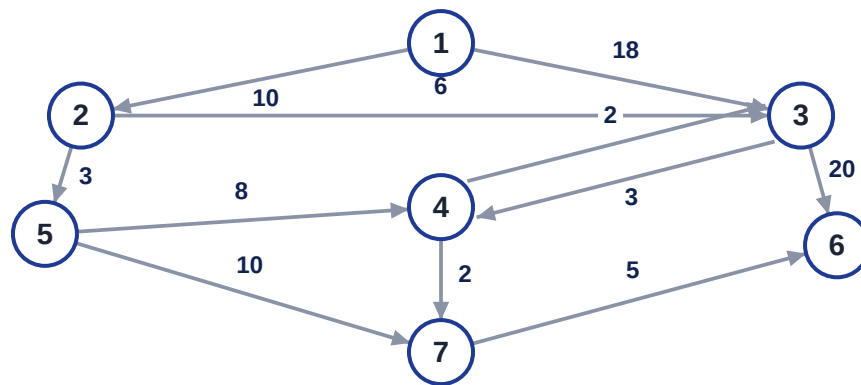


Figura 1. Dígrafo ponderado de la guía (7 vértices). Los números sobre las aristas son sus pesos.

La tabla siguiente muestra el vector $\text{dist}[]$ tras cada extracción (en negrita el vértice que se fija en ese paso). El símbolo - representa infinito.

Paso	Fija	d1	d2	d3	d4	d5	d6	d7
0	—	0	-	-	-	-	-	-
1	1	0	10	18	-	-	-	-
2	2	0	10	16	-	13	-	-
3	5	0	10	16	21	13	-	23
4	3	0	10	16	19	13	36	23
5	4	0	10	16	19	13	36	21
6	7	0	10	16	19	13	26	21
7	6	0	10	16	19	13	26	21

Resultado final: distancias mínimas desde el nodo 1 = (0, 10, 16, 19, 13, 26, 21).

Obsérvese un punto clave: para llegar al vértice 3, la arista directa 1 → 3 cuesta 18, pero el rodeo 1 → 2 → 3 cuesta 16. El algoritmo descubre que el camino mínimo no siempre es la arista directa.

7. Demostración de correctitud

Se demuestra que cuando un vértice u se extrae de la cola, su distancia ya es definitiva: $\text{dist}[u] = \text{delta}(s, u)$.

Invariante del algoritmo

En todo momento, para cada vértice ya extraído u se cumple $\text{dist}[u] = \text{delta}(s, u)$, y para cada vértice v aún en la cola, $\text{dist}[v]$ es el costo del camino mínimo de s a v que usa únicamente vértices ya extraídos como intermedios.

Esquema de la prueba (por contradicción). Supóngase que u es el primer vértice extraído para el cual $\text{dist}[u] > \text{delta}(s, u)$. Sea P un camino mínimo real de s a u , y sea (x, y) la primera arista de P que sale de la región de vértices ya fijados (x fijado, y todavía en la cola). Como x se fijó correctamente, al relajar (x, y) se tuvo $\text{dist}[y] \leq \text{delta}(s, x) + w(x, y) = \text{delta}(s, y)$. Dado que los pesos son no negativos y y aparece antes de u en P , se cumple $\text{delta}(s, y) \leq \text{delta}(s, u) < \text{dist}[u]$. Entonces $\text{dist}[y] < \text{dist}[u]$, por lo que el algoritmo habría extraído y antes que u , contradiciendo la elección de u . Por tanto $\text{dist}[u] = \text{delta}(s, u)$.

El papel de los pesos no negativos

La desigualdad $\text{delta}(s, y) \leq \text{delta}(s, u)$ depende crucialmente de que ningún peso sea negativo: una arista negativa posterior podría abaratar un camino ya fijado, rompiendo el invariante. Por eso Dijkstra no admite pesos negativos, y el simulador los rechaza en la interfaz.

8. Análisis de complejidad

El costo depende de la estructura usada para la cola de prioridad y de la operación de extracción del mínimo. Sean $n = |V|$ vértices y $m = |E|$ aristas:

Implementación de la cola	Extraer mínimo	Relajar	Complejidad total
Arreglo / búsqueda lineal	$O(n)$	$O(1)$	$O(n^2)$
Heap binario (montículo)	$O(\log n)$	$O(\log n)$	$O((n + m) \log n)$
Heap de Fibonacci	$O(\log n)$	$O(1)$ amortizado	$O(m + n \log n)$

La guía clásica describe la versión con arreglo, de complejidad $O(n^2)$, adecuada para grafos pequeños o densos como los del simulador. Para grafos grandes y dispersos, la versión con heap binario, $O((n + m) \log n)$, es muy superior.

9. Caminos hacia un destino (grafo inverso)

El simulador ofrece, además del modo clásico “desde un origen”, un modo “hacia un destino”: calcular el camino mínimo desde **cada** vértice hacia un vértice destino t fijo. Teóricamente equivale a ejecutar Dijkstra sobre el **grafo transpuesto** (o inverso) G^T , en el que cada arista (u, v) se reemplaza por (v, u) , tomando t como origen.

Formalmente, la distancia de v a t en G es igual a la distancia de t a v en G^T . En la práctica, basta con relajar las aristas **entrantes** a cada vértice en lugar de las salientes; el código implementa exactamente esta

variante.

10. Comparación: Bellman-Ford y Floyd-Warshall

Dijkstra no es el único algoritmo de caminos mínimos. La elección depende del problema:

Algoritmo	Problema	Pesos negativos	Complejidad
Dijkstra	Fuente única	No	$O(n^2)$ u $O((n+m) \log n)$
Bellman-Ford	Fuente única	Sí (detecta ciclos negativos)	$O(n \cdot m)$
Floyd-Warshall	Todos los pares	Sí (sin ciclos negativos)	$O(n^3)$

Bellman-Ford admite pesos negativos y detecta ciclos de costo negativo, a cambio de mayor tiempo. **Floyd-Warshall** es un algoritmo de **programación dinámica** que obtiene las distancias entre todos los pares de vértices; su idea central es permitir progresivamente a cada vértice k actuar como intermedio: $d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$. La guía propone añadir Floyd-Warshall como complemento al simulador.

11. Aplicaciones y relación con la implementación

El problema del camino mínimo aparece en navegación GPS y mapas, enrutamiento de paquetes en redes (protocolos tipo OSPF), redes de transporte y logística, videojuegos (búsqueda de rutas), y planificación de proyectos, entre otros.

En el simulador, la teoría se refleja directamente en el código:

- El grafo se guarda como listas de *nodos* y *aristas*; se puede importar/exportar como matriz de adyacencia con la convención -1 = sin arista.
- La función `dijkstra` implementa el bucle de selección del mínimo y la relajación descritos en la sección 6.
- El vector `prev` (predecesores) permite reconstruir las rutas, gracias a la subestructura óptima.
- El rechazo de pesos negativos en la interfaz garantiza las hipótesis del teorema de correctitud.
- El modo “hacia un destino” aplica la idea del grafo transpuesto de la sección 9.

12. Glosario y referencias

Glosario

Vértice / nodo	Elemento del grafo; punto de la red.
Arista / arco	Conexión entre dos vértices; en grafos ponderados lleva un peso.
Peso	Costo asociado a una arista (distancia, tiempo, dinero).
$\delta(s, v)$	Costo del camino mínimo entre s y v .
Relajación	Operación que mejora la estimación de distancia de un vértice.

Algoritmo voraz	Estrategia que toma en cada paso la opción localmente óptima.
SSSP / APSP	Camino mínimo de fuente única / entre todos los pares.

Referencias

- [1] Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs*. Numerische Mathematik, 1, 269-271.
- [2] Cormen, T., Leiserson, C., Rivest, R. y Stein, C. *Introduction to Algorithms*, 3.ª ed., MIT Press (caps. de caminos mínimos: Dijkstra, Bellman-Ford y Floyd-Warshall).
- [3] Bellman, R. (1958). *On a routing problem*. Quarterly of Applied Mathematics.
- [4] Floyd, R. W. (1962). *Algorithm 97: Shortest Path*. Communications of the ACM.
- [5] Guía No. 10 — Programación IV: *Algoritmos para la ruta más corta en un grafo*.